EXPERIMENT 3

Token Ring Algorithm:

token_ring.py Python

```python
import threading
import time

class Node:

    def __init__(self, node_id):
        self.node_id = node_id
        self.next_node = None
        self.has_token = False

    def set_next_node(self, node):
        self.next_node = node

    def receive_token(self):
        self.has_token = True

    def process(self):
        while True:

            if self.has_token:

                print(f"Node {self.node_id} is using the token (critical section)")
                time.sleep(2)

                print(f"Node {self.node_id} passing token to Node
                {self.next_node.node_id}")

                self.has_token = False
                self.next_node.receive_token()

            time.sleep(1)


# Creating nodes
node1 = Node(1)
node2 = Node(2)
node3 = Node(3)
node4 = Node(4)

# Creating the ring
node1.set_next_node(node2)
node2.set_next_node(node3)
node3.set_next_node(node4)
node4.set_next_node(node1)

# Initial token holder
node1.has_token = True

# Running nodes using threads
threads = []

for node in [node1, node2, node3, node4]:
    t = threading.Thread(target=node.process)
    threads.append(t)
    t.start()
```

Output:

```
PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS    OUTPUT                zsh - expt_3

➜  expt_3 git:(main) ✗ python3 token_ring.py
Node 1 is using the token
Node 1 passing token to Node 2
Node 2 is using the token
Node 2 passing token to Node 3
Node 3 is using the token
Node 3 passing token to Node 4
Node 4 is using the token
Node 4 passing token to Node 1
Node 1 is using the token
Node 1 passing token to Node 2
Node 2 is using the token
Node 2 passing token to Node 3
Node 3 is using the token
Node 3 passing token to Node 4
Node 4 is using the token
Node 4 passing token to Node 1

Token Ring Simulation Finished
➜  expt_3 git:(main) ✗
```

Bully Algorithm:

bully.py Python

```python
class Process:

    def __init__(self, pid):
        self.pid = pid
        self.alive = True

def bully_election(processes, initiator):

    print(f"\nProcess {initiator.pid} starts the election")

    higher = [p for p in processes if p.pid > initiator.pid and p.alive]
    if not higher:
        print(f"Process {initiator.pid} becomes the new coordinator")
        return initiator
    for p in higher:
        print(f"Process {initiator.pid} sends election message to Process {p.pid}")
    new_initiator = max(higher, key=lambda x: x.pid)

    return bully_election(processes, new_initiator)

# Creating processes
processes = [
    Process(1),
    Process(2),
    Process(3),
    Process(4),
    Process(5)
]

# Simulating leader failure
processes[-1].alive = False

# Starting election
leader = bully_election(processes, processes[1])
print(f"\nLeader elected: Process {leader.pid}")

```

Output:

PROBLEMS DEBUG CONSOLE TERMINAL PORTS OUTPUT zsh - expt_3

```
➜  expt_3 git:(main) ✗ python3 bully.py

Process 2 starts the election
Process 2 sends election message to Process 3
Process 2 sends election message to Process 4

Process 4 starts the election
Process 4 becomes the new coordinator

Leader elected: Process 4
➜  expt_3 git:(main) ✗
```